

Started on Thursday, 23 February 2023, 10:02 AM**State** Finished**Completed on** Thursday, 23 February 2023, 11:18 AM**Time taken** 1 hour 15 mins**Grade** Not yet graded**Question 1**

Correct

Mark 1.00 out of 1.00

Assign the numerically equal values from different number systems to each other

- | | | |
|---|-------------------------------|---|
| the hexadecimal number 0A is equal to | the decimal number 10 | ✓ |
| the binary sequence 1000 0000 is equal to | the decimal number 128 | ✓ |
| the hexadecimal number C3 is equal to | the binary sequence 1100 0011 | ✓ |
| the binary sequence 0011 1111 is equal to | the hexadecimal number 3F | ✓ |
| the decimal number 255 is equal to | the hexadecimal number FF | ✓ |
| the decimal number 9 is equal to | the binary sequence 0000 1001 | ✓ |

Die Antwort ist richtig.

The correct answer is: the hexadecimal number 0A is equal to → the decimal number 10, the binary sequence 1000 0000 is equal to → the decimal number 128, the hexadecimal number C3 is equal to → the binary sequence 1100 0011, the binary sequence 0011 1111 is equal to → the hexadecimal number 3F, the decimal number 255 is equal to → the hexadecimal number FF, the decimal number 9 is equal to → the binary sequence 0000 1001

Question 2

Correct

Mark 1.00 out of 1.00

extend the 5-bit signed binary number 11001 by 3 bits, so that its value does not change and enter the new binary sequence

Answer: 11111001 ✓

The correct answer is: 11111001

Question 3

Correct

Mark 1.00 out of 1.00

when performing a bitwise AND of the inputs a and b, the following code can be used:

Select one:

- ☐ a. (a && b)
- ☐ b. (a | b)
- ☒ c. (a & b) ✓
- ☐ d. (a || b)

Die Antwort ist richtig.

The correct answer is: (a & b)

Question 4

Correct

Mark 1.00 out of 1.00

to create an 8-bit vector of type logic, the following lines of code are correct

Select one or more:

- ☒ a. logic[0:7] myvar; ✓
- ☒ b. logic[1:8] myvar; ✓
- ☒ c. logic[7:0] myvar; ✓
- ☐ d. logic[8] myvar;

Die Antwort ist richtig.

The correct answers are: logic[7:0] myvar,, logic[0:7] myvar,, logic[1:8] myvar;

Question 5

Correct

Mark 1.00 out of 1.00

Which statements about synthesizable and non-synthesizable code are true?

assume x and y are variable input signals

Select one or more:

- ☐ a. the mathematical function $y = z / x$ is synthesizable
- ☒ b. non-synthesizable code can contain time delay statements ✓
- ☒ c. the mathematical function $x = x + 1$ is synthesizable ✓
- ☐ d. the mathematical function $y = \log_2(x)$ is synthesizable

Die Antwort ist richtig.

The correct answers are: non-synthesizable code can contain time delay statements, the mathematical function $x = x + 1$ is synthesizable

Question 6

Correct

Mark 1.00 out of 1.00

Vivado will automatically detect the hierarchy of HDL files, due to the hierarchical modeling within the files (instantiation of sub-modules).

Select one:

- ☒ True ✓
- ☐ False

The correct answer is 'True'.

Question 7

Correct

Mark 1.00 out of 1.00

```
logic A,B,C,D;  
always @(posedge clk)  
begin  
  A <= din;  
  B <= A;  
  C <= B;  
  D <= C;  
end
```

The above code will create the following logic:

Select one:

- ☐ a. a direct connection from input to output, without any memory elements
- ☒ b. 4 flip flops connected in series ✓
- ☐ c. 4 latches connected in series

Die Antwort ist richtig.

The correct answer is: 4 flip flops connected in series

Question 8

Correct

Mark 1.00 out of 1.00

Which statements about flip-flops are true?

Select one or more:

- ☐ a. A flip-flop will sample data immediately whenever the input data changes
- ☐ b. A flip-flop will only store data temporarily and needs to be refreshed after some time
- ☐ c. flip-flop is just another name for latch.
- ☒ d. A flip-flop will only sample the input value on an active edge of the clock signal ✓

Die Antwort ist richtig.

The correct answer is: A flip-flop will only sample the input value on an active edge of the clock signal

Question 9

Correct

Mark 1.00 out of 1.00

What are the following system functions used for?

- | | | |
|--|--|---|
| \$monitor ("format_string", par_1, par_2, ...); | prints the string every time one of the parameters changes | ✓ |
| \$finish | halts the simulation | ✓ |
| \$display ("format_string", par_1, par_2, ...); | prints the string to the console when called | ✓ |

Die Antwort ist richtig.

The correct answer is: \$monitor ("format_string", par_1, par_2, ...); → prints the string every time one of the parameters changes, \$finish → halts the simulation, \$display ("format_string", par_1, par_2, ...); → prints the string to the console when called

Question 10

Correct

Mark 1.00 out of 1.00

Which statements about verilog parameters are true?

Select one or more:

- ☒ a. parameters can be used as constants ✓
- ☒ b. parameters can be used to modify the vector size of signals ✓
- ☒ c. a parameter defined within a modules interface can be overwritten by the instantiating top level module ✓
- ☒ d. just like input and output signals, a SystemVerilog module can have parameters within it's interface. ✓
- ☐ e. parameters can change during the execution of HDL code.

Die Antwort ist richtig.

The correct answers are: parameters can be used to modify the vector size of signals, just like input and output signals, a SystemVerilog module can have parameters within it's interface., a parameter defined within a modules interface can be overwritten by the instantiating top level module, parameters can be used as constants

Question 11

Correct

Mark 1.00 out of 1.00

How can an asynchronous input be synchronized to the core clock?

Select one:

- ☐ a. With a counter circuit
- ☐ b. with a demultiplexer
- ☐ c. with a toggle flip-flop
- ☒ d. with a shift register ✓

Die Antwort ist richtig.

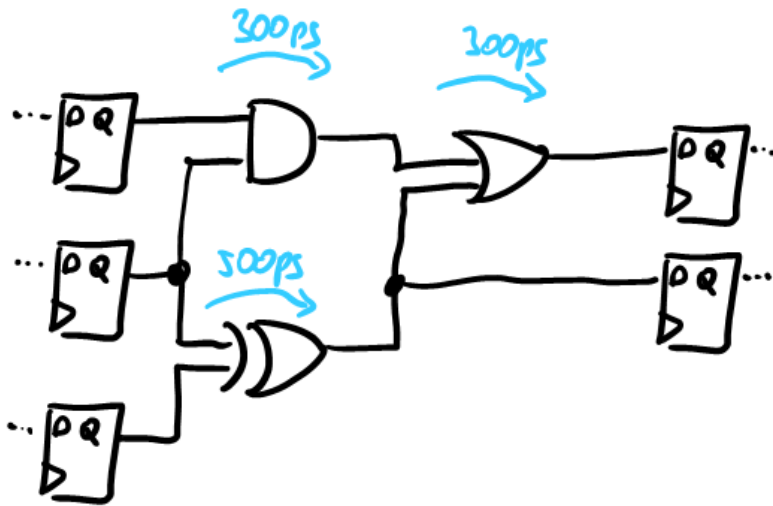
The correct answer is: with a shift register

Question 12

Correct

Mark 1.00 out of 1.00

Consider the following section of a sequential circuit:



Which statements about its internal combinational paths are true?

The input to output delay of each gate is given in blue.

- ☒ a. The combinational paths begin at the flip-flop outputs and end at the flip-flop inputs. ✓
- ☒ b. the longest (critical) path delay is 800ps ✓
- ☐ c. The shortest combinational path delay is 300ps
- ☐ d. the longest (critical) path delay is 600ps
- ☒ e. the shortest combinational path delay is 500ps ✓

Your answer is correct.

The correct answers are: The combinational paths begin at the flip-flop outputs and end at the flip-flop inputs., the shortest combinational path delay is 500ps,

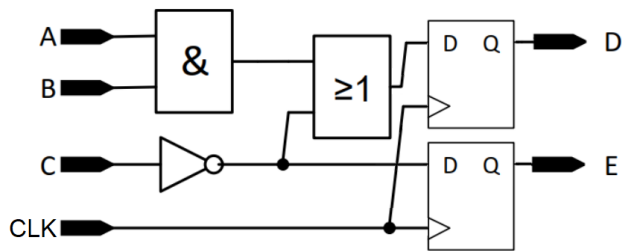
the longest (critical) path delay is 800ps

Question 13

Complete

Marked out of 3.00

Describe a module in SystemVerilog with the ports and functionality given in the schematic below:



```

module(
input logic A,
input logic B,
input logic C,
input logic CLK,
output logic D,
output logic E
);
logic k, d, f, g;
assign k = A & B;
assign d = !C;
assign g = k + d;
always_comb begin
if(g >= 1) begin
f = 1;
end
else begin
f = 0;
end
end
always_ff @ (posedge CLK) begin
if (f) begin
D <= f;
end
else begin
D <= 0;
end
end
always_ff @ (posedge CLK) begin
if (d) begin
E <= d;
end
else begin

```



```
D <=0;  
end  
end  
endmodule
```

Question 14

Complete

Marked out of 3.00

What are timing violations?

And how can they be mitigated if they do occur?

In case of input voltage supplied to a flipflop, lies between a valid high and low, a data is captured at the output. it goes into a metastability situation. such an event is called timing violation.

Mitigation can be done by following points

- 1) reducing the system clock
- 2) reducing the combinational delay of the code (pipelined)

Question 15

Correct

Mark 1.00 out of 1.00

Which of the following statements performs a correct signed addition?

The following signal definitions are done

logic signed [7:0] a;

logic unsigned [7:0] b;

logic signed [8:0] sum;

Select one:

- ☐ a. assign sum = a + \$signed(b);
- ☐ b. assign sum = a + b;
- ☒ c. assign sum = a + \$signed({1'b0, b}); ✓

Die Antwort ist richtig.

The correct answer is: assign sum = a + \$signed({1'b0, b});

Question 16

Correct

Mark 1.00 out of 1.00

Which type of reset is used for the sequential element "deb_dout_ff" of the following RTL code?

```
always_ff @(posedge clk) begin
    if (reset_i) begin
        deb_dout_ff <= 1'b0;
    end else begin
        deb_dout_ff <= deb_dout_val;
    end
end
```

Select one:

- ☒ a. Active high synchronous reset ✓
- ☐ b. Active low synchronous reset
- ☐ c. Active low asynchronous reset
- ☐ d. Active high asynchronous reset

Die Antwort ist richtig.

The correct answer is:

Active high synchronous reset

Question 17

Correct

Mark 1.00 out of 1.00

How many Flip Flops are generated with the following RTL code in case the parameter DATA_W=8?

```
// -----
// Definition
// -----
localparam logic [DATA_W-1:0] zero = {(DATA_W) {1'b0}};

// delayed output value
logic [DATA_W-1:0] ynl_ff;

logic [DATA_W-1:0] avg_ff;

logic [DATA_W:0] sum, sum1 , sum2;

// -----
// Implementation
// -----

// Shift register
always_ff @(posedge clk or negedge reset_ni) begin
    if (~reset_ni) begin
        ynl_ff <= {(DATA_W) {1'b0}};
    end else begin
        if (avg_clr_i) begin
            ynl_ff <= {(DATA_W) {1'b0}};
        end else if (avg_en_i) begin
            // _INFO_ This works because unsigend values are used
            ynl_ff <= sum[DATA_W-1:0];
        end
    end
end

// Adder
// a = 0.125
// 1-a = 0.875;
// _INFO_ There is no multiplier used!
assign sum1 = (din_i >> 3);
assign sum2 = (ynl_ff >> 3) + (ynl_ff >> 2) + (ynl_ff >> 1);
assign sum = sum1 + sum2;

// Output register plus divide by 4
always_ff @(posedge clk or negedge reset_ni) begin
    if (~reset_ni) begin
        avg_ff <= {(DATA_W) {1'b0}};
    end else begin
        avg_ff <= sum[DATA_W-1:0];
    end
end
assign dout_o = avg_ff;
```

Answer: 16



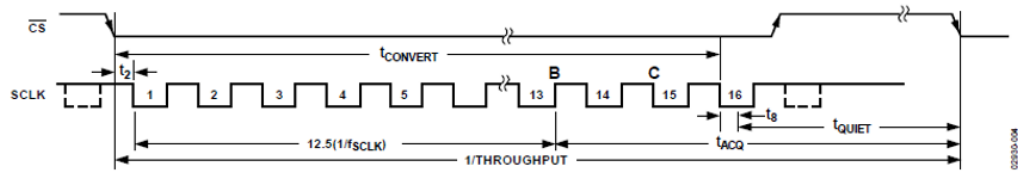
The correct answer is: 16

Question 18

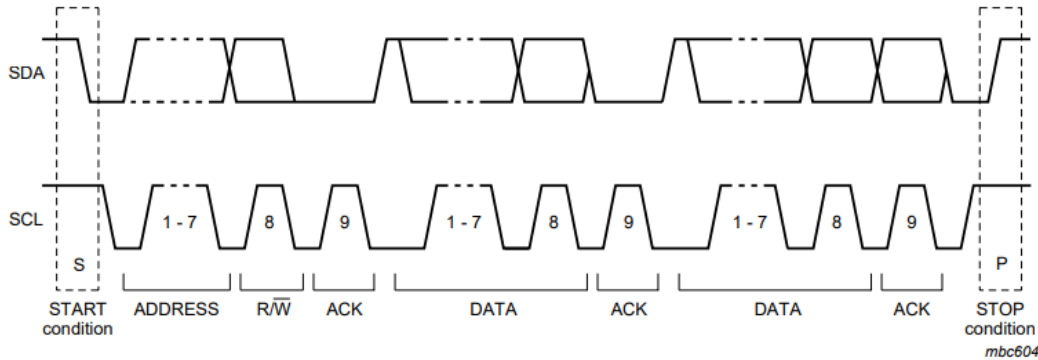
Correct

Mark 1.00 out of 1.00

Assign the following interface timing diagrams to the correct interface type.



SPI interface timing

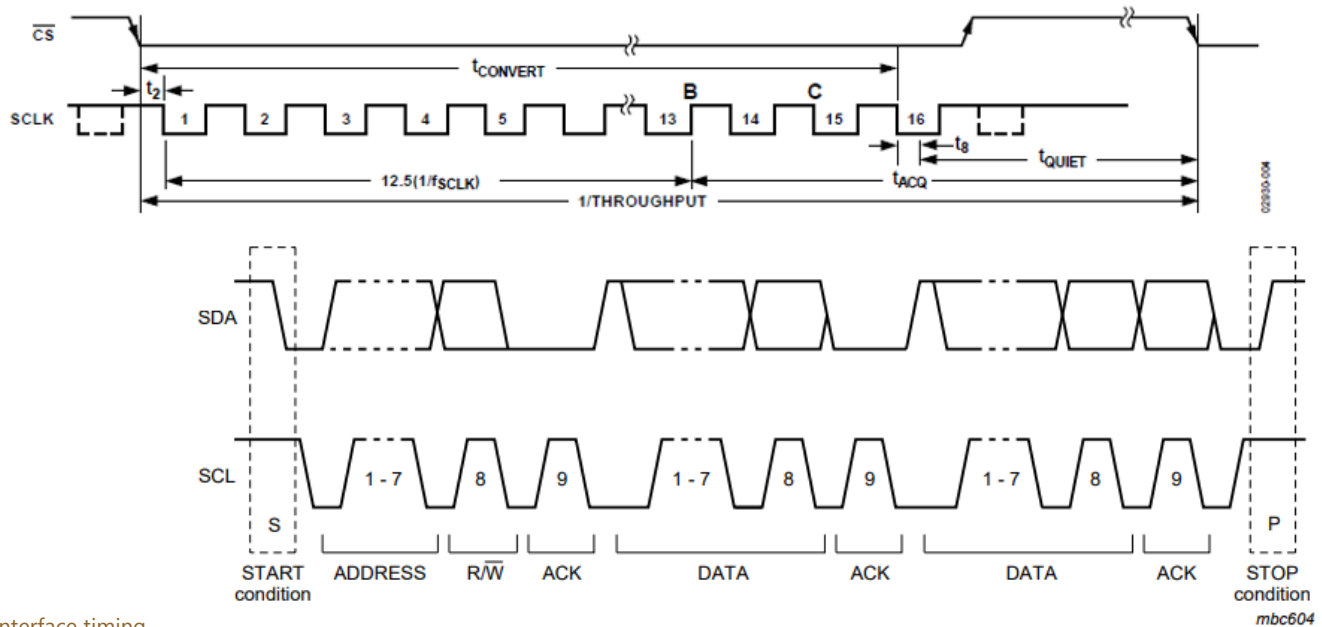


I2C interface timing



Die Antwort ist richtig.

The correct answer is:



SPI interface timing,
→ I2C interface timing

Question 19

Correct

Mark 1.00 out of 1.00

UART communication: What is the purpose of the parity bit.

Select one or more:

- ☐ a. Start of transfer synchronization
- ☒ b. Error detection mechanism for detecting a single bit errors ✓
- ☐ c. Error detection mechanism for correcting single bit errors

Die Antwort ist richtig.

The correct answer is: Error detection mechanism for detecting a single bit errors

Question 20

Correct

Mark 1.00 out of 1.00

The data of a SPI based system is transferred synchronous to the clock signal.

Select one:

- ☒ True ✓
- ☐ False

The correct answer is 'True'.

Question 21

Correct

Mark 1.00 out of 1.00

An I2C controller performs a data transfer of two bytes from the controller to the target (write transfer).
How many bits (including start and stop bit) need to be transferred for this data transfer.

Answer: ✓

The correct answer is: 29

Question 22

Correct

Mark 1.00 out of 1.00

An I2S interface requires the following pins:

Select one or more:

- ☒ a. Word select (WS) ✓
- ☒ b. Serial clock (SCK) ✓
- ☐ c. Chip select (CS)
- ☒ d. Serial data (SD) ✓

Die Antwort ist richtig.

The correct answers are: Serial clock (SCK), Word select (WS),
Serial data (SD)

Question 23

Correct

Mark 1.00 out of 1.00

The number of data bits used for a an SPI transfer is always 8 bit.

Select one:

- ☐ True
- ☒ False ✓

The correct answer is 'False'.

Question 24

Correct

Mark 1.00 out of 1.00

Assign the correct SystemVerilog code to the given statements .

The following SystemVerilog code generates an FPGA output called "top_pin".



The following SystemVerilog code generates an FPGA input called "top_pin".



The following SystemVerilog code generates an FPGA output called "top_pin" which can be switched to tristate.



The following SystemVerilog code generates an FPGA in /output called "top_pin".



Die Antwort ist richtig.

The correct answer is: The following SystemVerilog code generates an FPGA output called "top_pin". → assign top_pin = sig;, The following SystemVerilog code generates an FPGA input called "top_pin". → assign sig = top_pin;, The following SystemVerilog code generates an FPGA output called "top_pin" which can be switched to tristate. → assign top_pin = (en) ? sig : 1'bz;, The following SystemVerilog code generates an FPGA in /output called "top_pin". → assign top_pin = (en) ? sig : 1'bz; assign sigin = top_pin;

Question 25

Correct

Mark 1.00 out of 1.00

Inside an FPGA Combinational Logic is implemented with Look-up Tables (LUT)

Select one:

- ☒ True 
- ☐ False

The correct answer is 'True'.

Question 26

Incorrect

Mark 0.00 out of 1.00

An Double Data Rate Flip Flop generates an new output value only at the falling edge of the clock.

Select one:

- ☒ True ❌
- ☐ False

The correct answer is 'False'.

Question 27

Correct

Mark 1.00 out of 1.00

At an AXI streaming interface a data transfer takes place if ...

Select one or more:

- ☐ ... always if the signal TREADY is high for two clock cycles
- ☐ ... the signals TVALID and TDATA are high
- ☐ ... the signal TVALID or the signal TREADY is high
- ☒ ... if the signal TREADY and the signal TVALID is high ✔️

Die Antwort ist richtig.

The correct answer is: ... if the signal TREADY and the signal TVALID is high

Question 28

Correct

Mark 1.00 out of 1.00

An APB Bus write access without wait states takes exactly two clock cycles.

Select one:

- ☒ True ✔️
- ☐ False

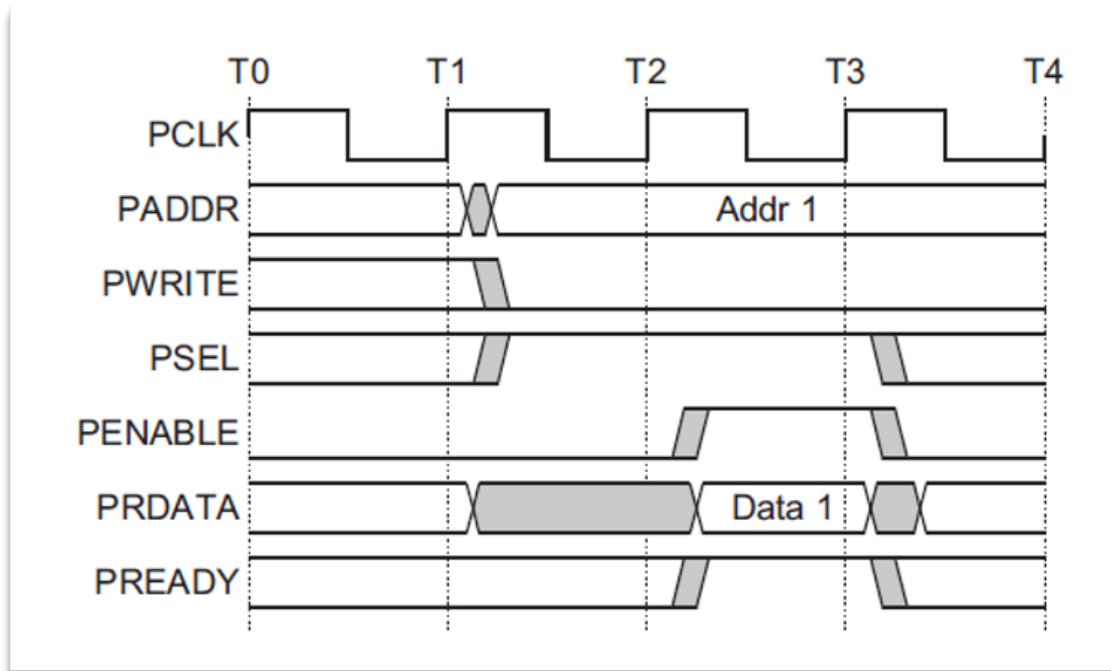
The correct answer is 'True'.

Question 29

Correct

Mark 1.00 out of 1.00

The following timing diagram shows an APB Bus read access.



Select one:

☒ True ✓

☐ False

The correct answer is 'True'.

Question 30

Complete

Marked out of 3.00

Implement the SystemVerilog RTL code for a module called "top" with the following pins:

Inputs:

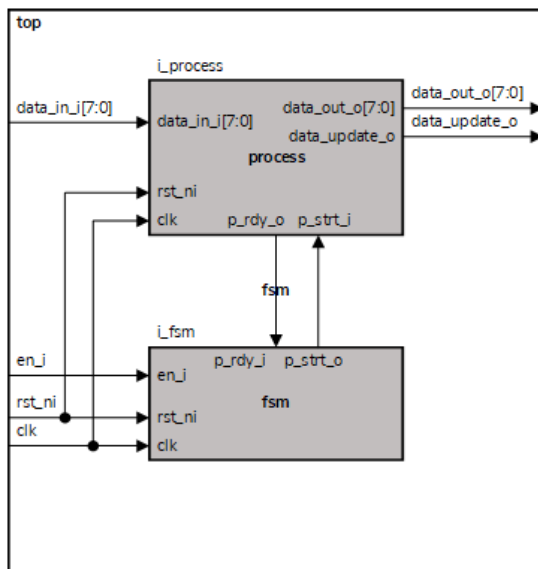
- rst_ni, clk, en_i, data_in_i[7:0]

Output:

- data_update_o, data_out_o[7:0]

Within the module "top" instantiate the modules "fsm" (instance name "i_fsm") and "process" (instance name "i_process") as shown in the block diagram.

It is not required to write the module definitions RTL code of module "fsm" and "process".



```

module top(
input logic [7:0] data_in_i,
input logic rst_ni,
input logic clk,
input logic en_i,
output logic [7:0] data_out_o,
output logic data_update_o
);
logic p_rdy_o, p_rdy_i;
logic p_strt_i, p_strt_o;

```

```

i_process DUT1 (
.data_in_i (data_in_i),
.rst_ni (rst_ni),
.clk (clk),
.en_i (en_i),
.data_out_o (data_out_o),
.data_update_o (data_update_o),

```

```
.p_rdy_o (p_rdy_i),  
.p_strt_i (p_strt_o)  
);  
i_fsm DUT2 (  
    .rst_ni (rst_ni),  
    .clk (clk),  
    .en_i (en_i),  
    .p_rdy_i (p_rdy_o),  
    .p_strt_o (p_strt_i)  
);  
endmodule
```

Question 31

Complete

Marked out of 3.00

Implement the SystemVerilog RTL code for a module called "counter" with the following functionality:

Inputs:

- reset_ni: active low asynchronous reset
- clk: clock
- cnt_clr_i: Counter clear
- cnt_en_i: Counter enable
- cnt_max_i [7:0]: Counter max value

Output:

- cnt_rdy_o

Reset: Asynchronous active low reset

Counter range: 0 ...255

Counter counts until the counter value equals input cnt_max_i. The counter counts with the rising clock edge (clk) and if cnt_en_i=1.

Counter output cnt_rdy_o=1 if counter value equals cnt_max_i.

Counter is cleared if input cnt_clr_i=1 (highest priority).

The inputs cnt_clr_i, cnt_en_i and cnt_max_i are synchronous to clk

```
module counter (
input logic reset_ni, \ active low asynchronous reset
input logic clk, \ clock
input logic cnt_clr_i, \ Counter clear
input logic cnt_en_i, \ Counter enable
input logic [7:0] cnt_max_i, \ Counter max value
output logic cnt_rdy_o
);
logic [7:0] cnt;
```

```
always_ff @ (posedge clk, negedge reset_ni) begin
if (!reset_ni) begin
cnt <= 0;
cnt_rdy_o <= 0;
end
else begin
else if (cnt_clr_i) begin
cnt <= 0;
cnt_rdy_o <= 0;
end
else if (cnt_en_i) begin
cnt <= cnt + 1;
if (cnt == cnt_max_i) begin
```

```
        cnt_rdy_o <= 1;
    end
else begin
    cnt_rdy_o <= 0;
end
end
end
end
endmodule
```

Question 32

Complete

Marked out of 3.00

Implement the SystemVerilog RTL code for a module called "pat_det" with the following functionality:

Inputs:

- reset_ni: active low synchronous reset
- clk: clock
- din_i: Input data signal
- en_i: Enable the functionality of the pattern detection

Output:

- pat_det_o: Pattern detected

Reset: Synchronous, active low

Pattern detection - If the value zero (0) is applied for 3 consecutive clock cycles to the input din_i the output pat_det_o is high (1) for one one clock cycle. After the detection of the pattern (3 consecutive 0) the detection starts again.

The pattern detection is active if input en_i = 1. Otherwise the output pat_det_o is always 0. If the input en_i changes from 0 to 1 (the module gets enabled) the pattern detection starts from the beginning again.

The input signal din_i is synchronous to the clock clk_i.

The input signal en_i is synchronous to the clock clk_i.

```
module pat_det (  
    input logic reset_ni,  
    input logic clk,  
    input logic din_i,  
    input logic en_i,  
    output logic pat_det_o  
);  
    logic [2:0] counter;  
  
    always_ff @ (posedge clk, negedge reset_ni) begin  
        if (din_i == 0) begin  
            counter <= 3d'3;  
            pat_det_o <= 0;  
        end  
        else if (en_i) begin  
            pat_det_o <= 0;  
        end  
        else begin  
            counter <= counter - 1;  
            if (counter == 0 && en_i == 1) begin  
                pat_det_o <= 1;  
            end  
            else begin  
                pat_det_o <= 0;  
            end  
        end  
    end  
endmodule
```